

Trabajo Práctico 2 — Mafia

[7507/9502] Algoritmos y Programación III
Primer cuatrimestre de 2026

Grupo | 7 |

Índice

1. Supuestos	2
2. Diagramas de clases	2
3. Diagramas de secuencia	4
3.1. Entrega 0	5
3.2. Entrega 1	6
3.3. Entrega 2	11
3.4. Entrega 3	16
3.5. Entrega 4	19
4. Diagramas de paquetes	21
5. Detalles de implementación	23
6. Detalle de Implementación	23
6.1. Partida	23
6.2. Registro de Eventos de la Partida	23
6.3. Estado Dinámico de los Jugadores	23
6.4. Resolución de Empates y Consensos	23
6.5. Bandos, Polimorfismo y Double Dispatch	24
6.6. Roles mediante Composición	24
6.7. Configuración Inicial	24
6.8. Fases del Ciclo de Juego	24
6.9. Interfaz Gráfica y Desacoplamiento (Observer y Command)	25
7. Excepciones	26

1. Supuestos

- El rol Padrino se presenta como un Ciudadano cuando es investigado por el Detective, pero su verdadero rol se revela al ser eliminado.
- En caso de empate durante la votación diurna, el resultado depende del criterio de empate configurado para la partida.
- Las acciones nocturnas se consideran simultáneas y se resuelven al finalizar la fase nocturna.
- La cantidad mínima de jugadores es de 5 y la máxima es de 12. Si se intenta iniciar una partida con una cantidad inválida de jugadores, el sistema lanza una excepción.
- Se asume que, si los miembros de la Mafia no logran alcanzar un consenso o se produce un empate en su votación nocturna, la noche finaliza sin ninguna víctima, en lugar de elegir una al azar.
- Se asume que, en caso de empate durante la votación diurna de los ciudadanos, se realiza una instancia de revotación (Ballotage) únicamente entre los jugadores empatados. Si el empate persiste, no se elimina a ningún jugador.
- Dado que se trata de un juego de identidades ocultas, se asume que los jugadores comparten la misma pantalla de manera presencial y no mira el monitor cuando el sistema indica que no es su turno o cuando un rol secreto está ejecutando su acción.
- Aunque el Médico actúa durante la fase nocturna, no puede visualizar la votación de la Mafia ni conocer el objetivo elegido.
- Del mismo modo, el Detective no puede visualizar la votación de la Mafia ni las acciones de los demás roles.
- En la fase diurna, la etapa de debate es opcional.
- Durante la votación diurna, un jugador puede votar por cualquier nominado válido, incluido él mismo.
- Tanto el Sheriff como el Detective pueden realizar investigaciones.
- El Sheriff y el Detective se modelan como dos roles distintos e independientes.
- Como máximo puede realizarse una única instancia de Ballotage por ronda; si luego de esta persiste el empate, no se elimina a ningún jugador.
- En caso de alcanzarse un consenso dentro de la Mafia, el resultado de la votación se determina según las reglas de prioridad establecidas por el modelo.

2. Diagramas de clases

En esta sección presentamos los diagramas de clases correspondientes al modelo planteado. A continuación, se presenta el diagrama de clases general del sistema.

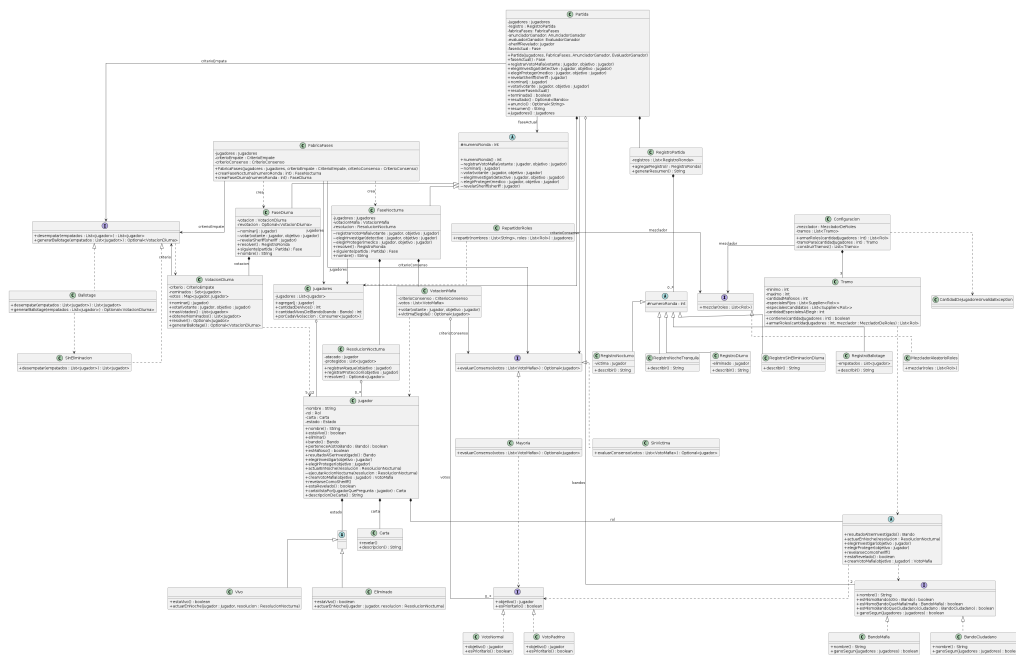


Figura 1: Diagrama de clases general del sistema.

Del mismo modo, presentamos el diagrama de clases correspondiente a los distintos roles del sistema.

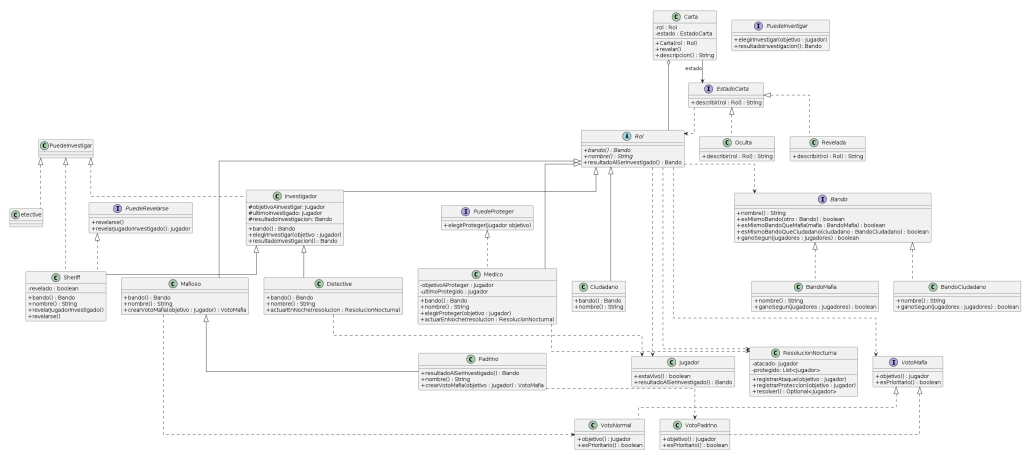


Figura 2: Diagrama de clases de los roles del sistema.

Por ultimo, presentamos el diagrama de excepciones.

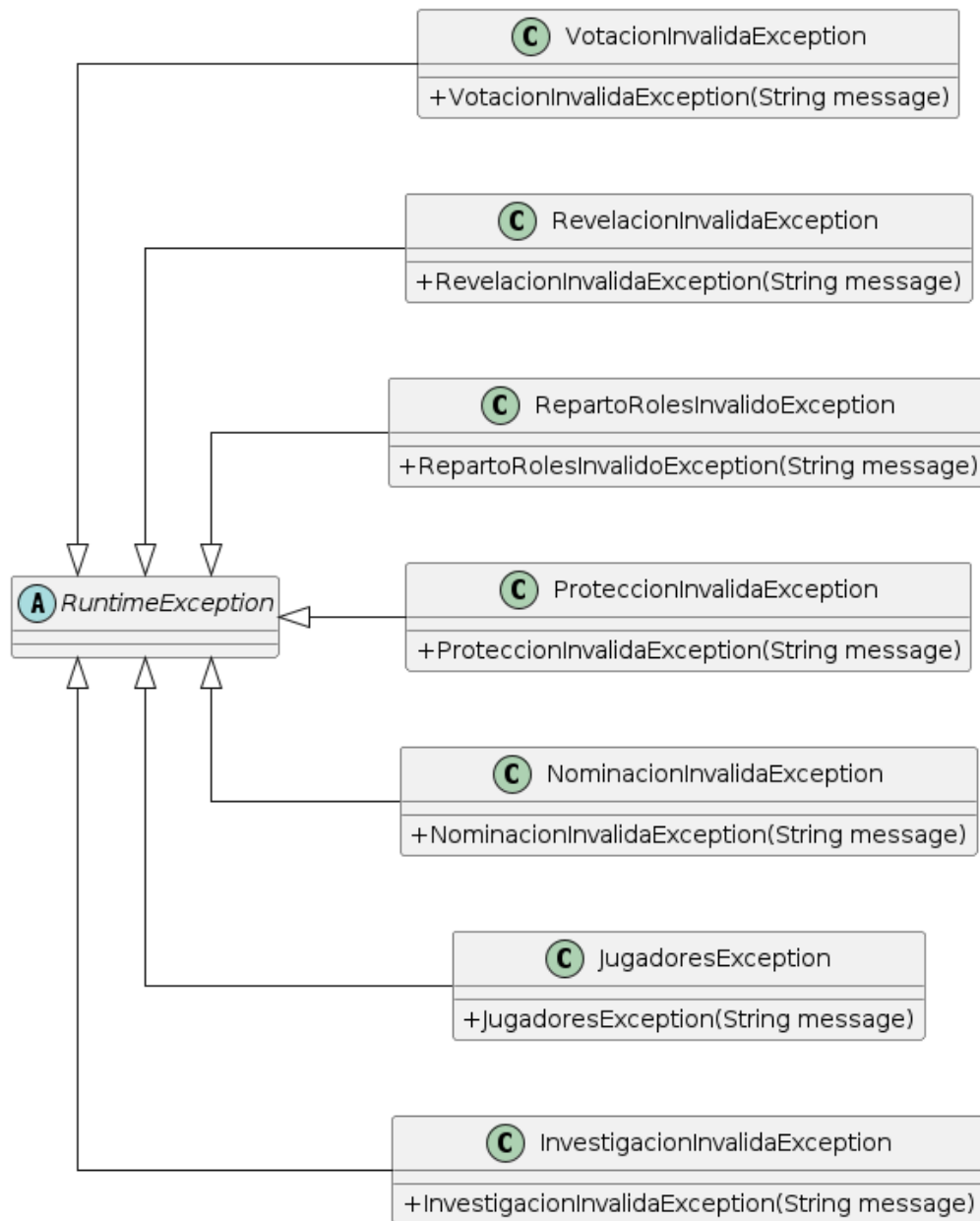


Figura 3: Diagrama de clases de los roles del sistema.

3. Diagramas de secuencia

En esta sección se ilustran distintos escenarios del juego que fueron mencionados en el enunciado del trabajo práctico, representados mediante diagramas de secuencia UML.

3.1. Entrega 0

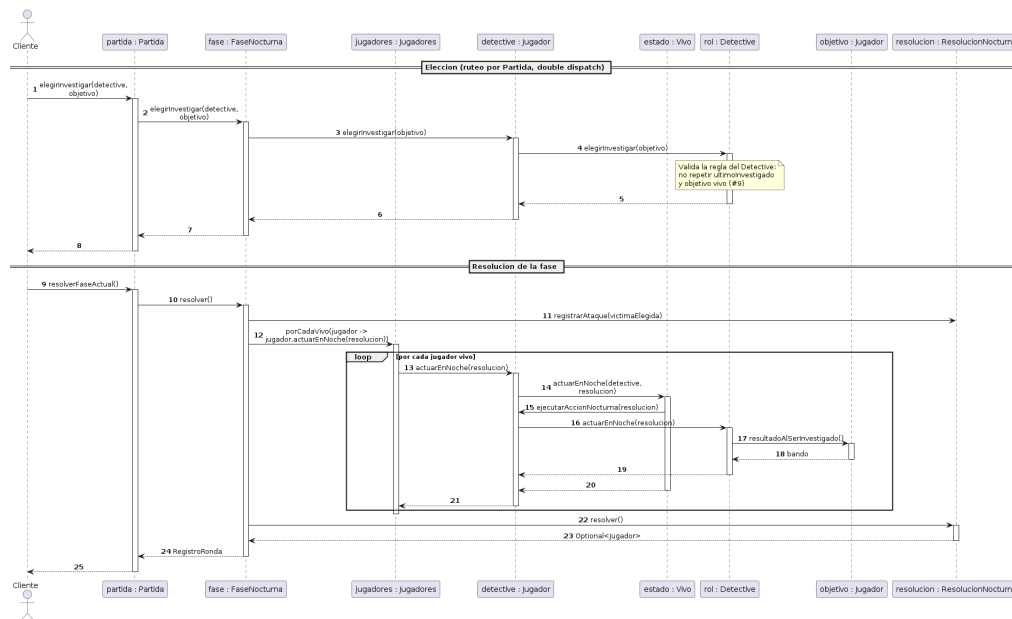


Figura 4: Diagrama de secuencia de ejecución de la fase nocturna completa.

3.2. Entrega 1

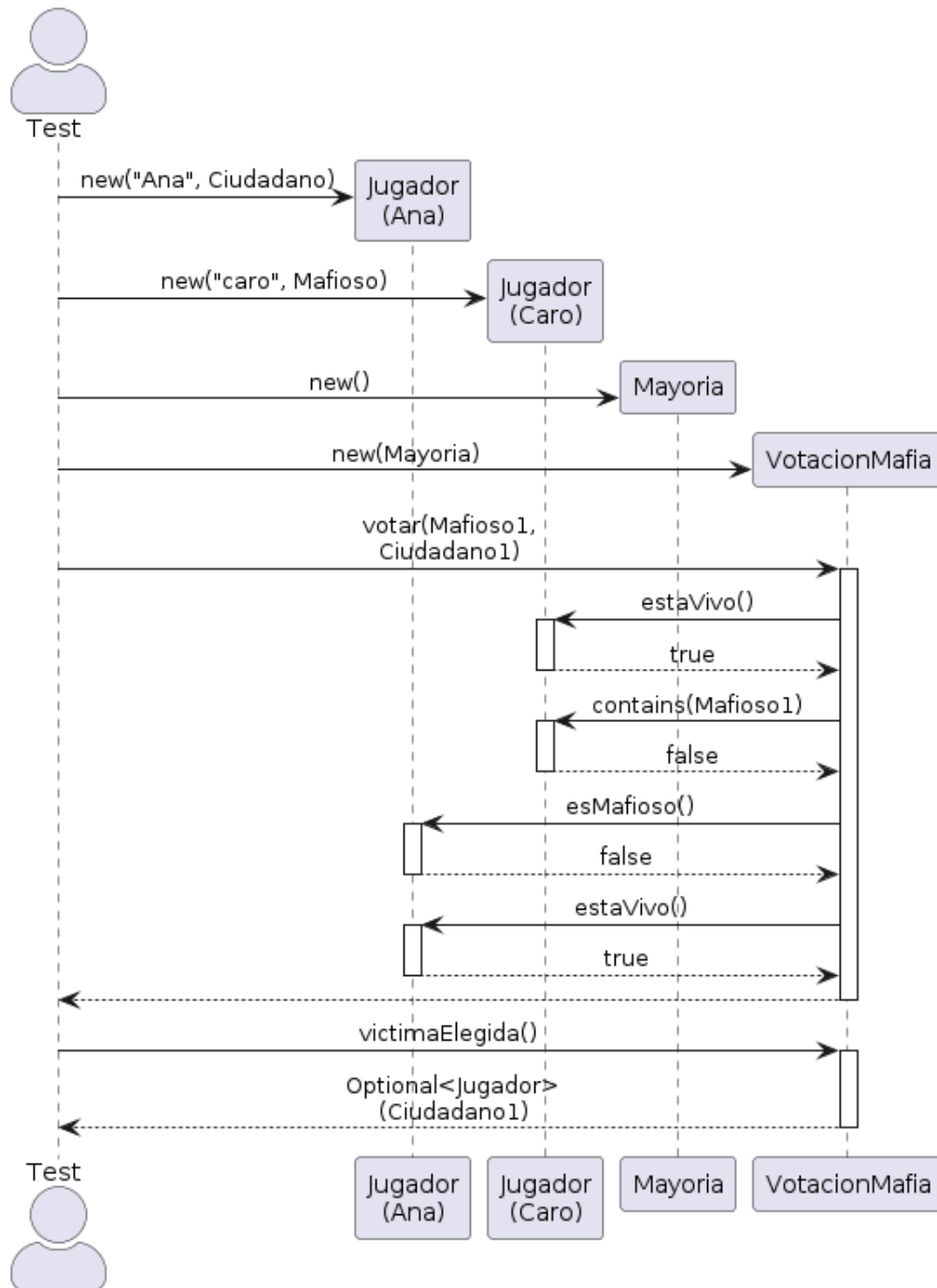


Figura 5: Diagrama de secuencia de la votacion mafia.

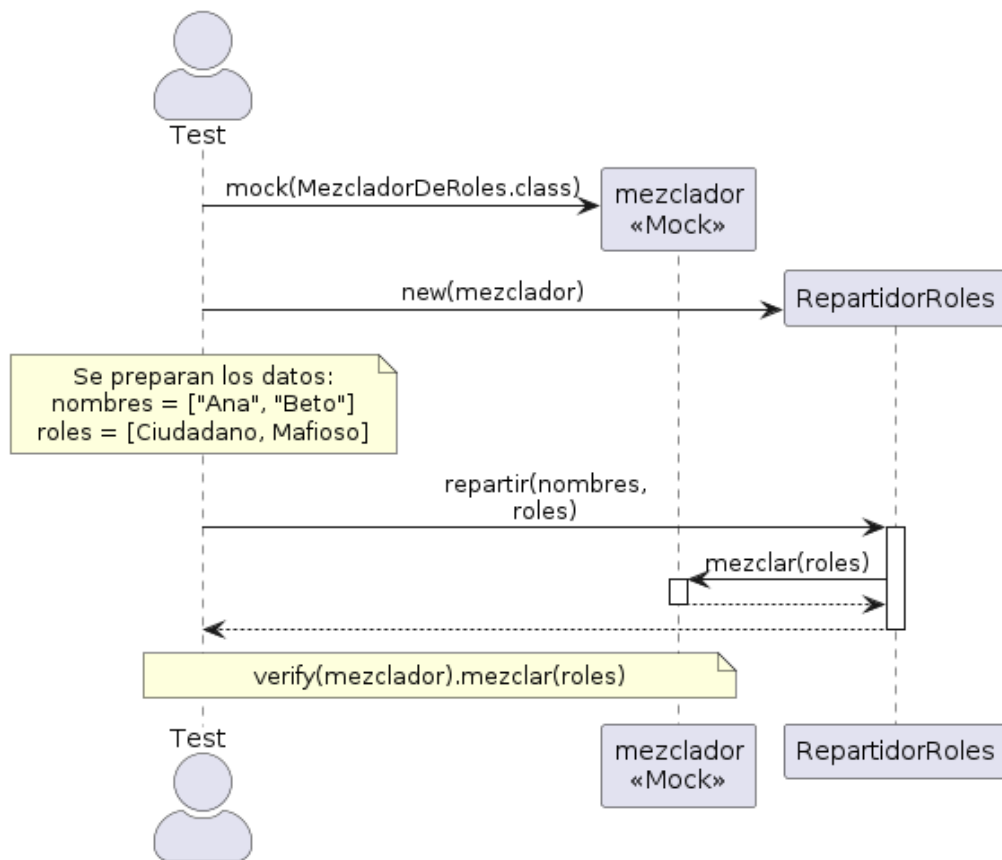


Figura 6: Diagrama de secuencia de la mezcla de roles.

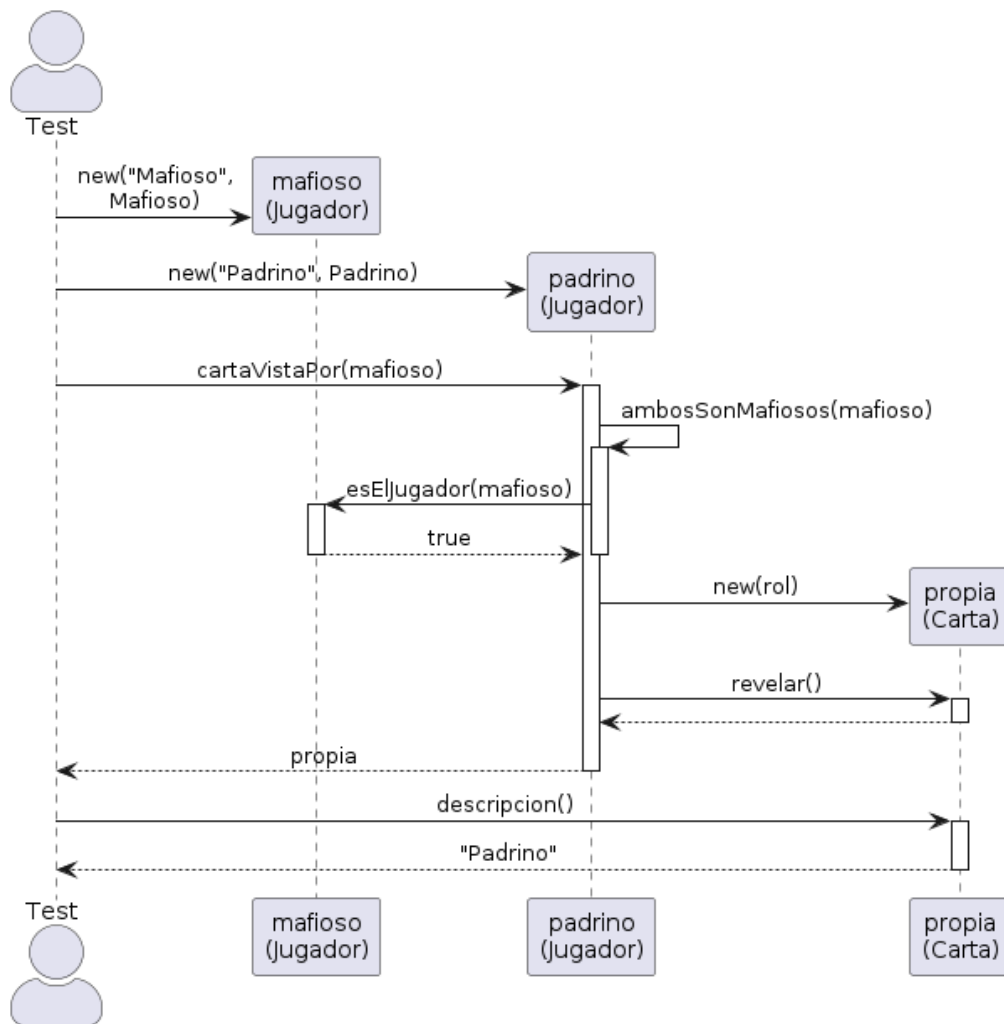


Figura 7: Diagrama de secuencia de la carta de padrino visto por mafioso.

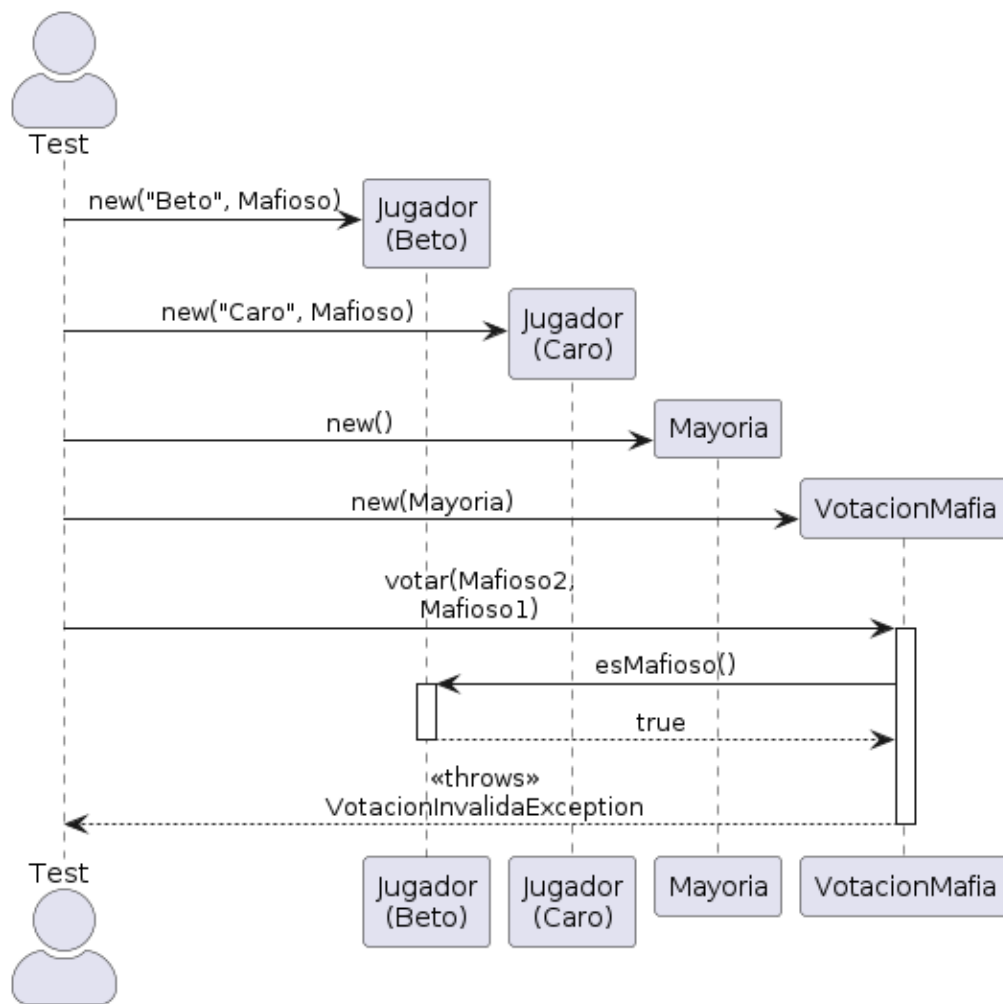


Figura 8: Diagrama de secuencia de lanzar excepcion cuando es votante no es valido .

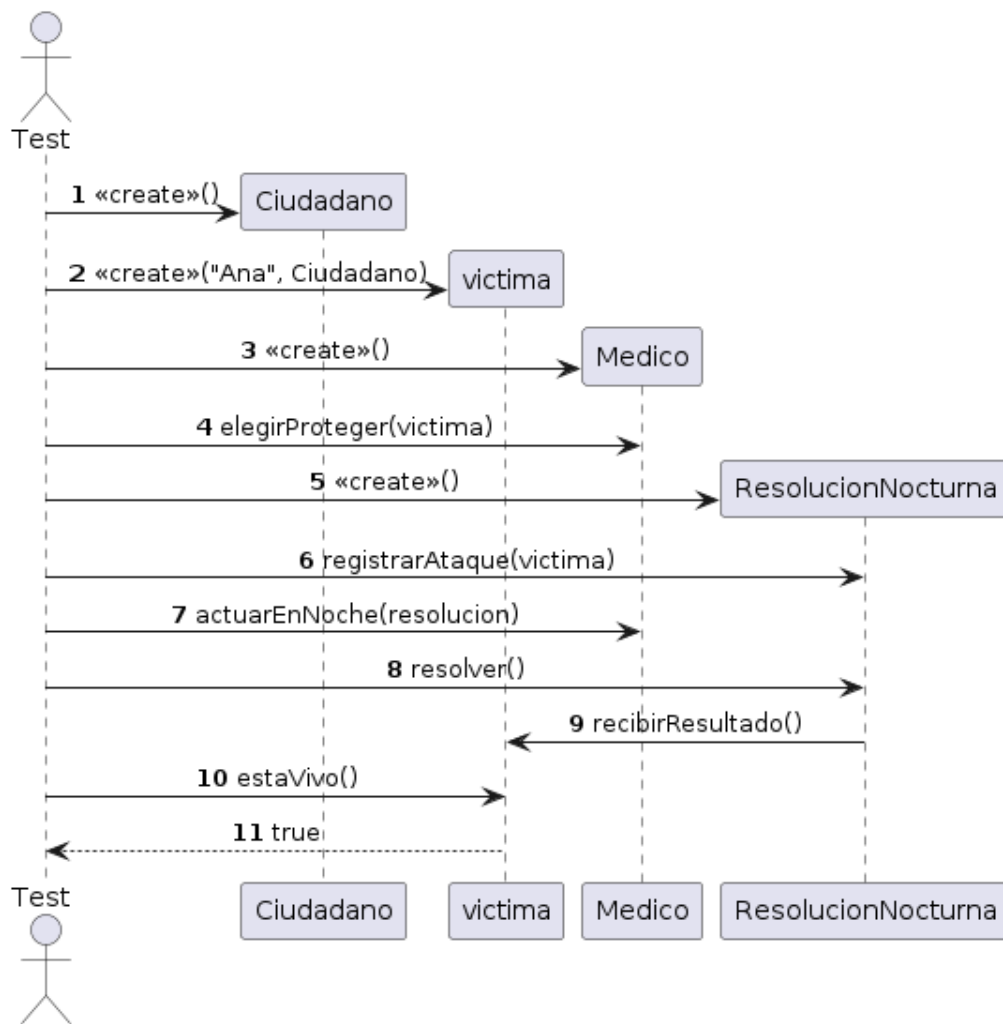


Figura 9: Diagrama de secuencia del proteccion de medico.

3.3. Entrega 2

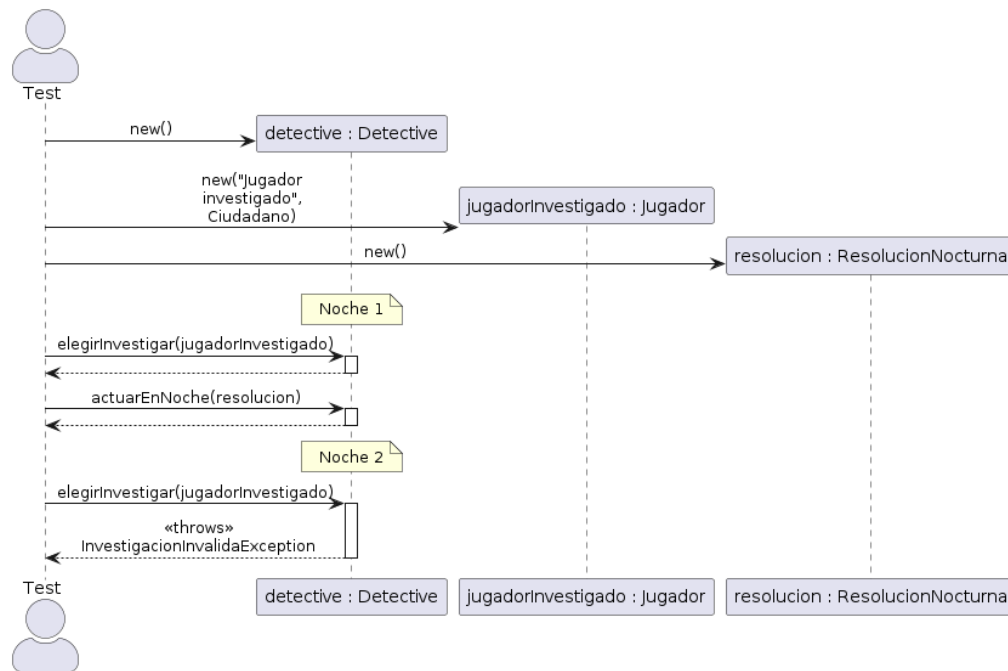


Figura 10: Diagrama de secuencia de detective investiga a un mafia.

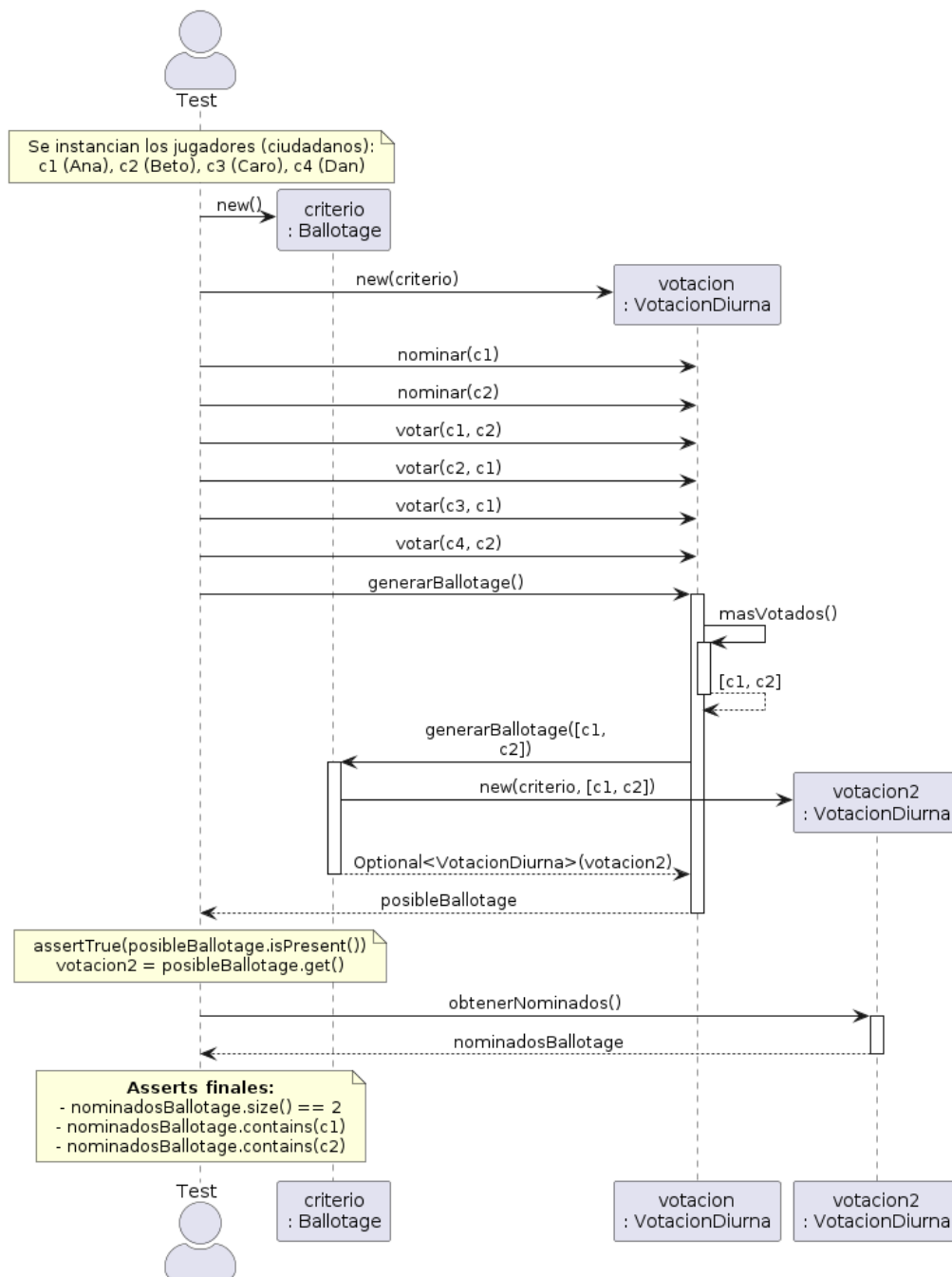


Figura 11: Diagrama de secuencia de votacion que entra en un ballotage.

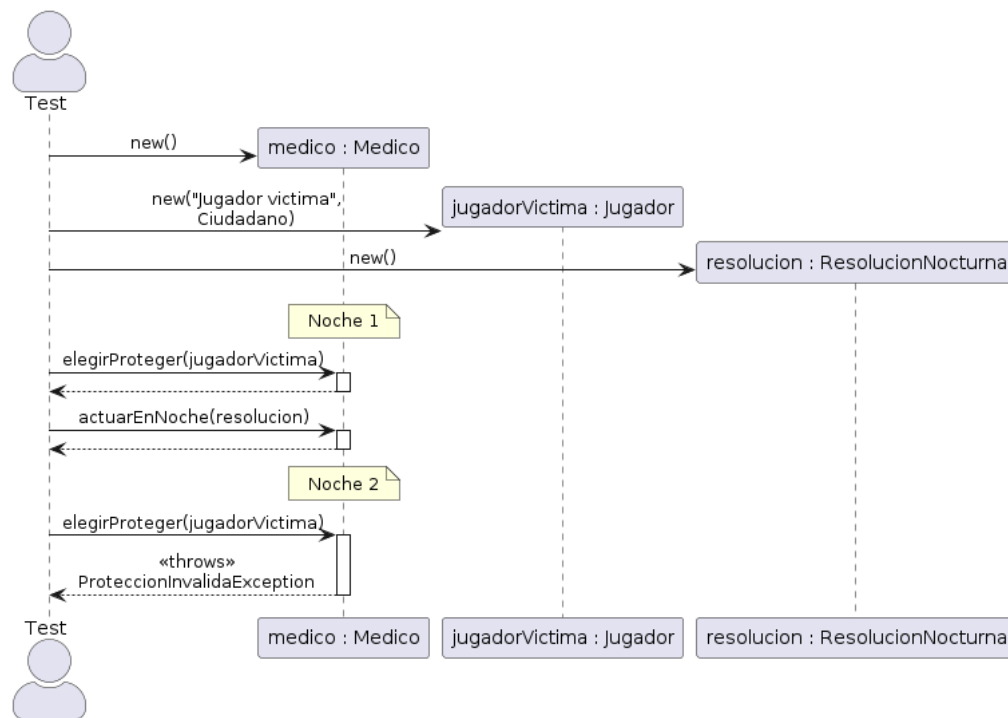


Figura 12: Diagrama de secuencia de proteccion doble.

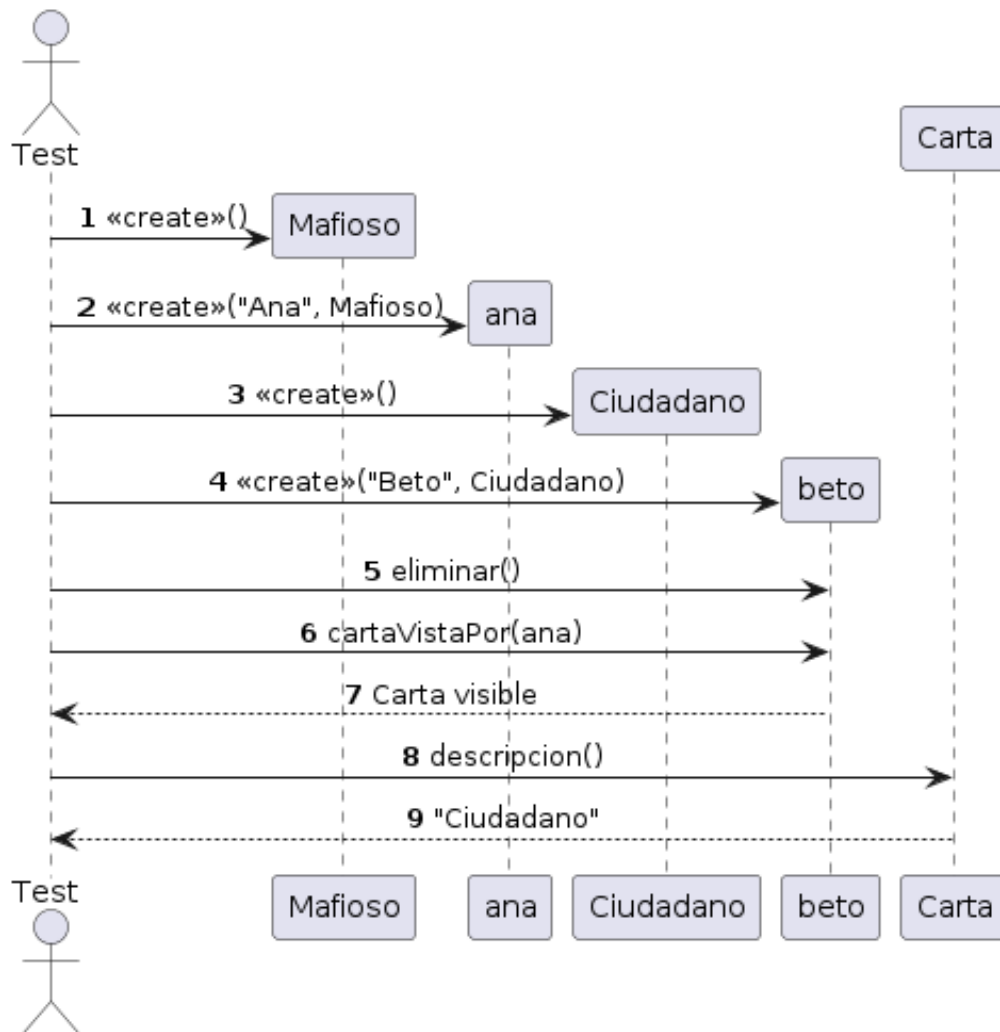


Figura 13: Diagrama de secuencia de la carta revelada.

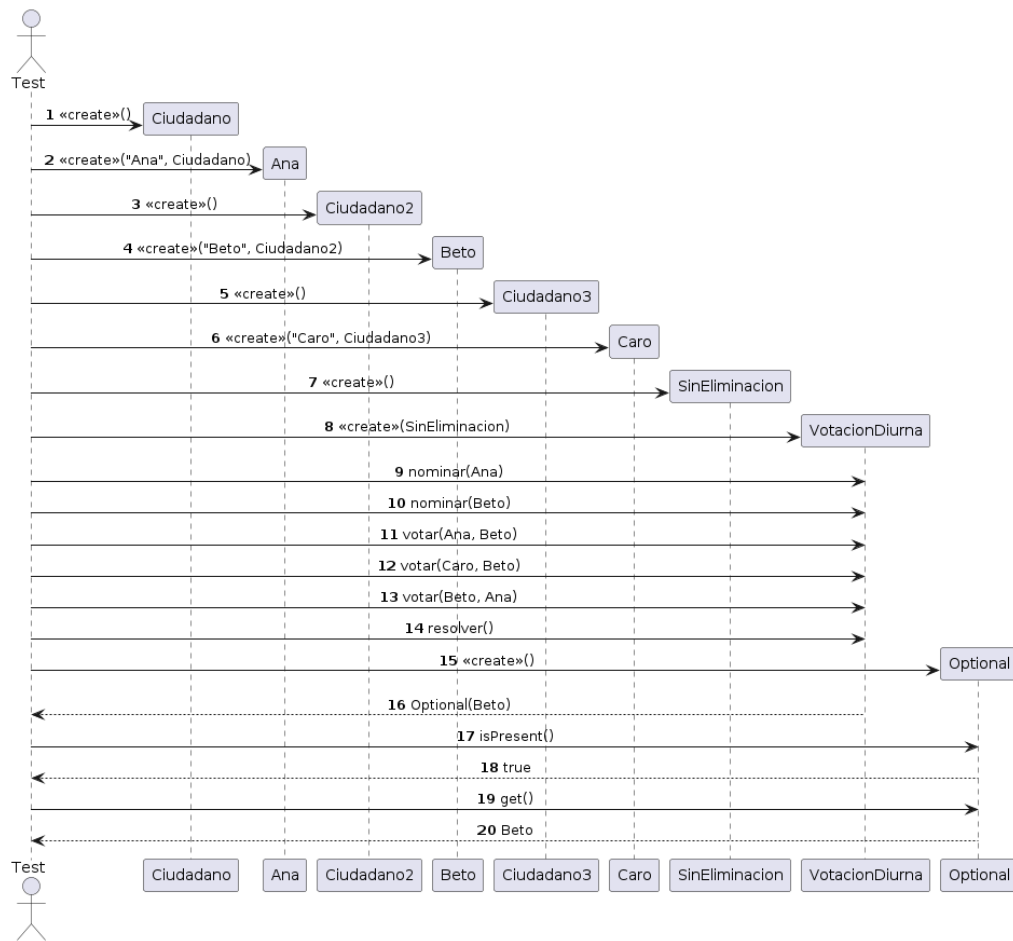


Figura 14: Diagrama de secuencia de votacion diurna.

3.4. Entrega 3

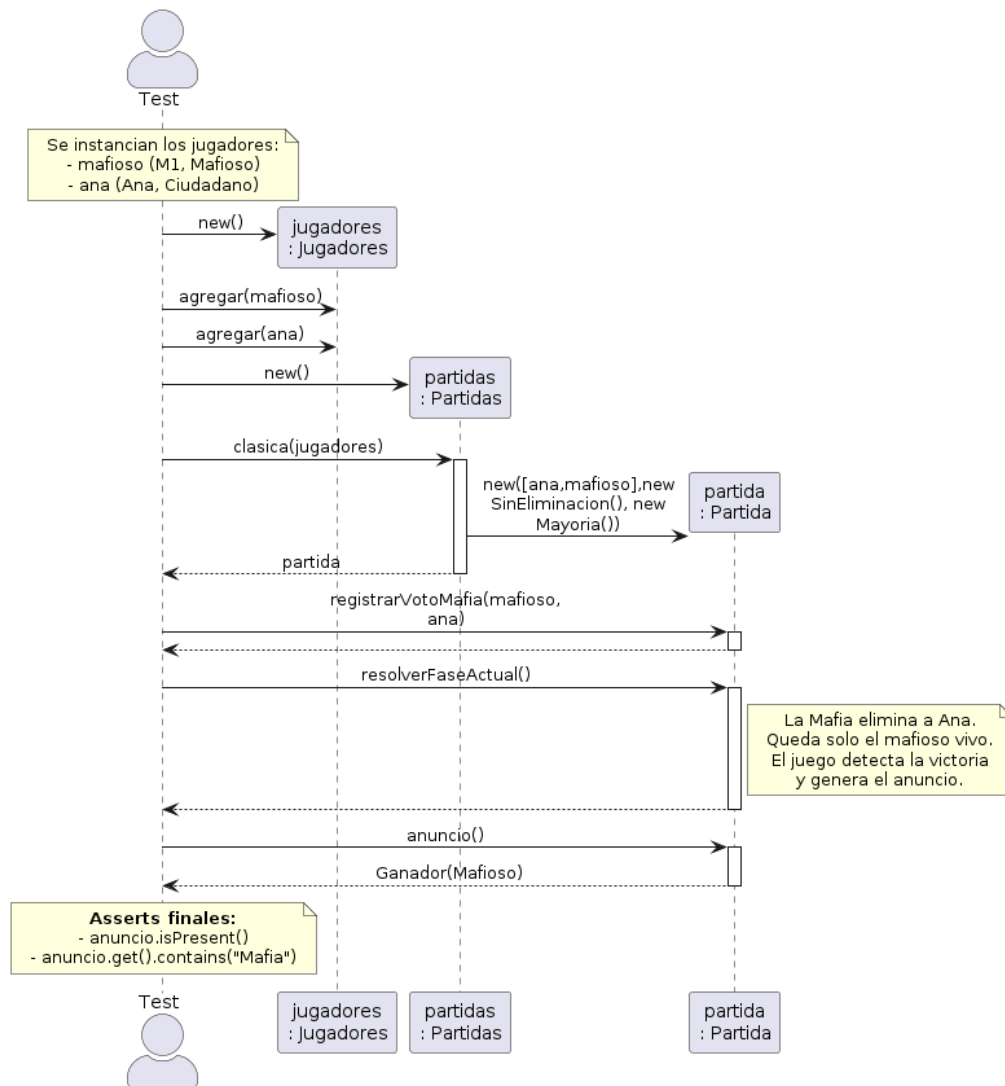


Figura 15: Diagrama de secuencia de anuncio de victoria.

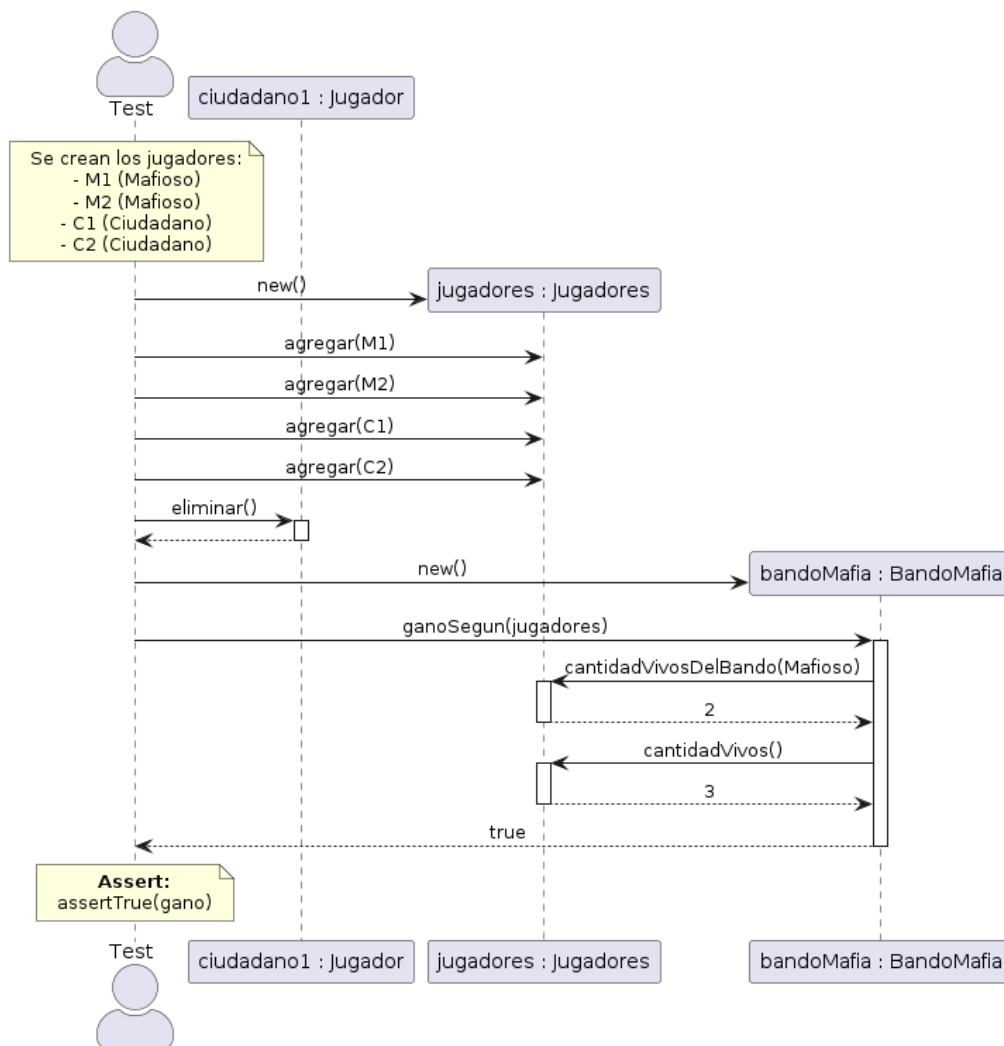


Figura 16: Diagrama de secuencia de condicion de victoria de mafia.

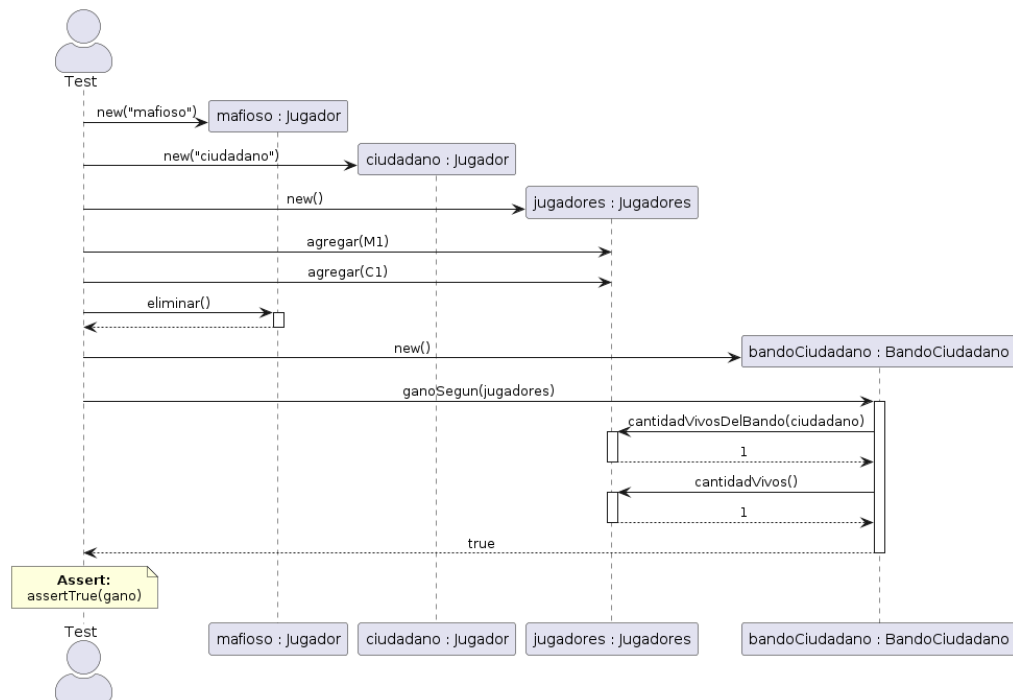


Figura 17: Diagrama de secuencia de condicion de victoria de ciudadano.

3.5. Entrega 4

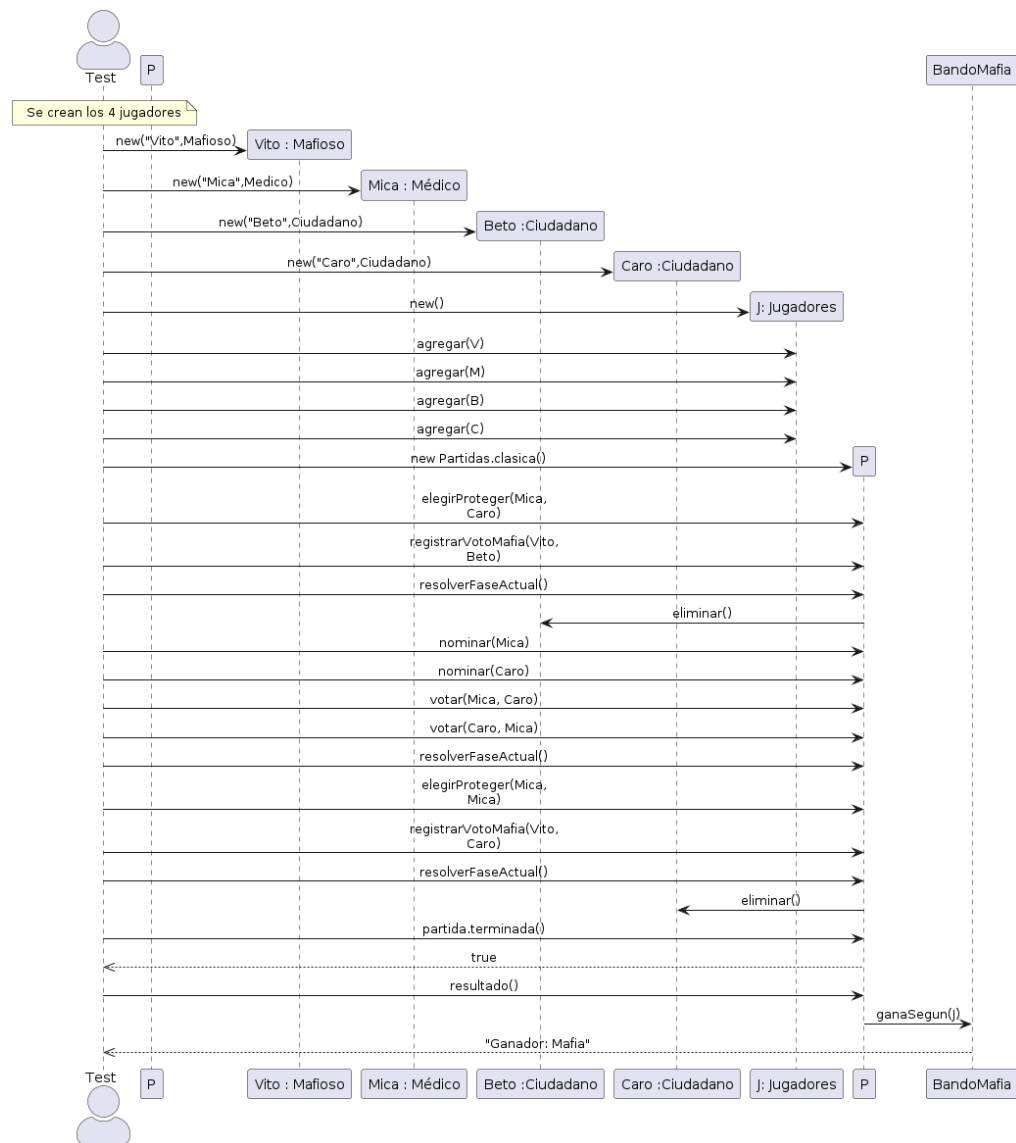


Figura 18: Diagrama de secuencia de la partida completa de que gana mafia.

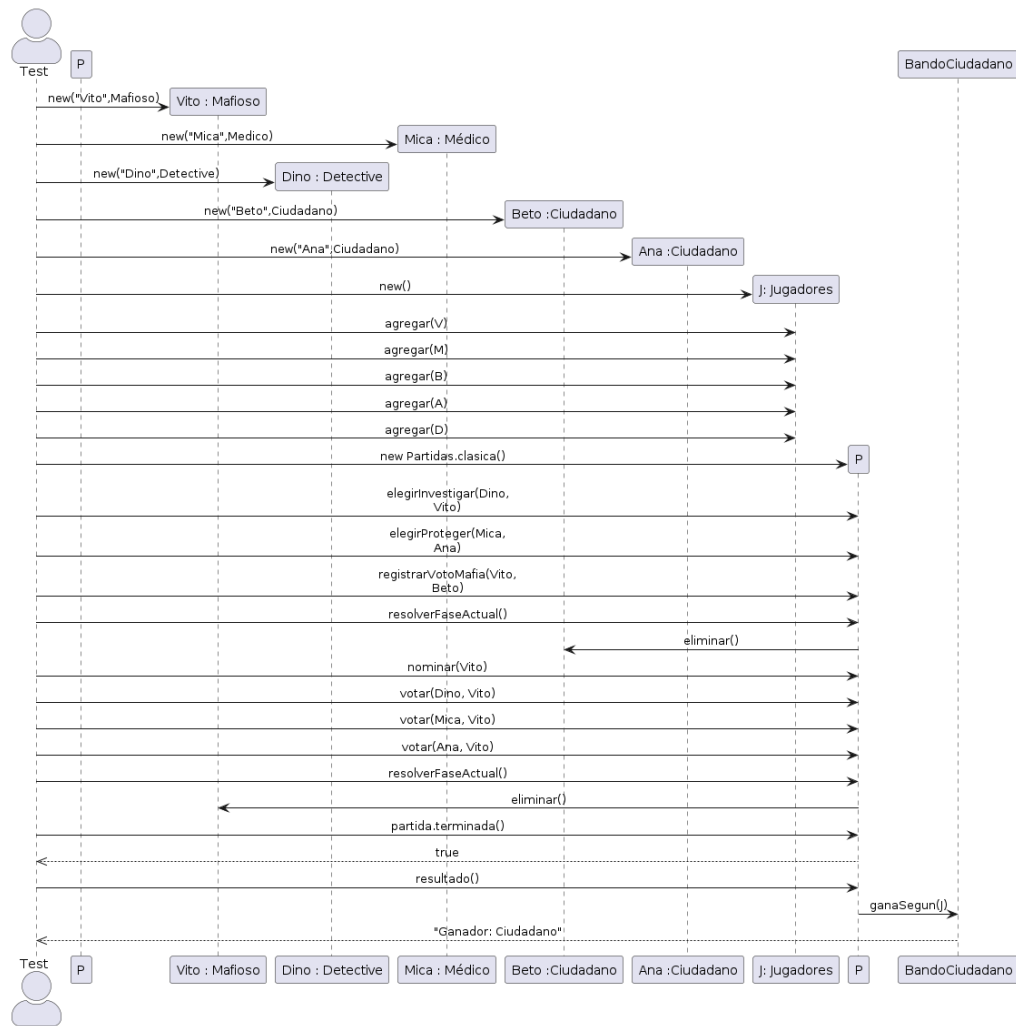


Figura 19: Diagrama de secuencia de la partida completa de que gana ciudadano.

4. Diagramas de paquetes

El sistema fue diseñado siguiendo el patrón arquitectónico MVC (Modelo–Vista–Controlador). En la implementación se desarrolló la capa de modelo, la cual contiene la lógica de negocio del juego, la capa de vista se encuentran previstas en el paquete vista y la capa de controlador incluido en el paquete de modelo por la particularidad del trabajo.

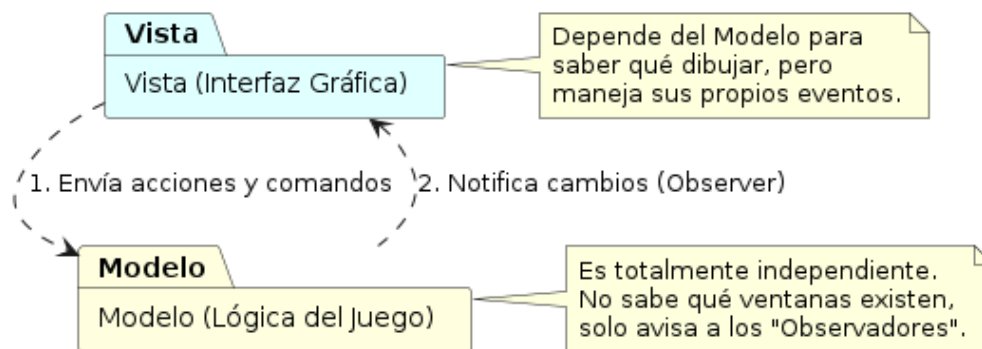


Figura 20: Diagrama de paquete.

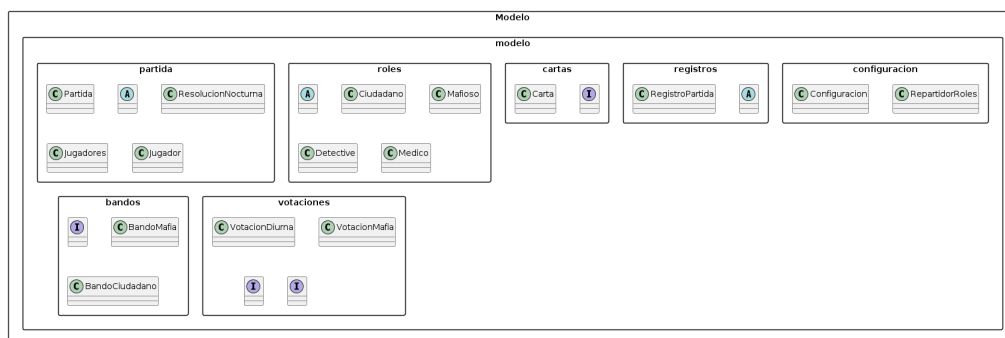


Figura 21: Diagrama de paquete del modelo.

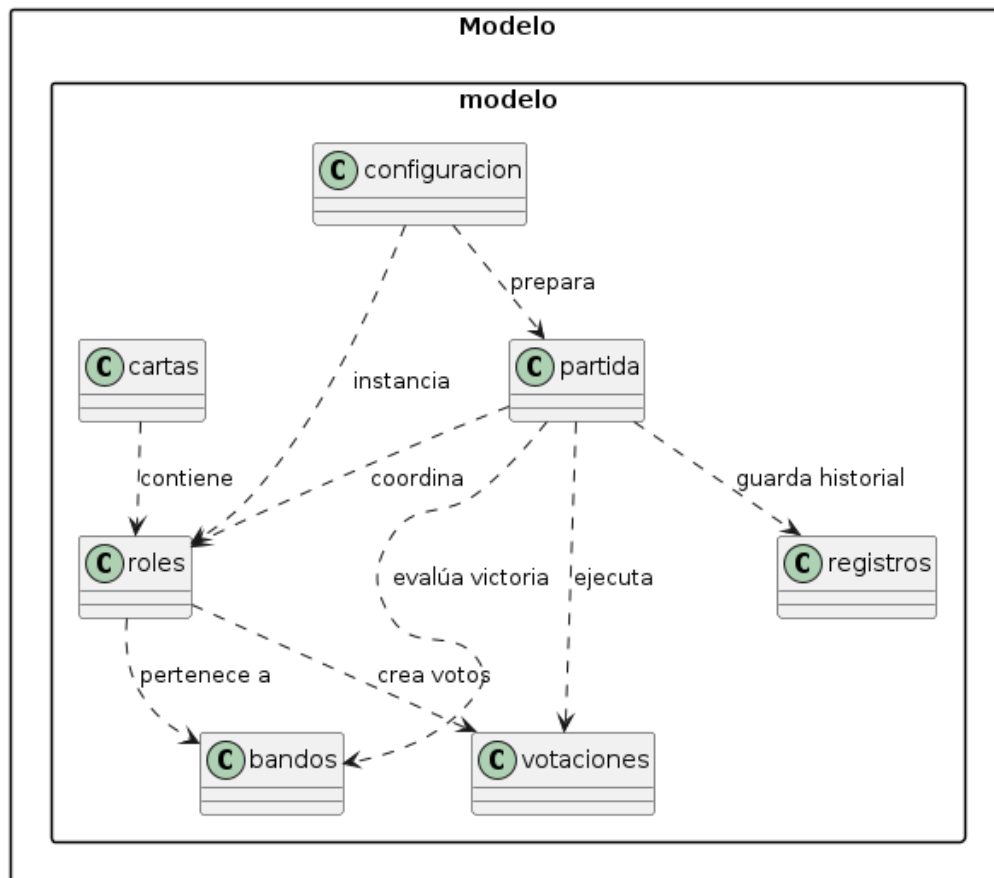


Figura 22: Diagrama de paquete de vista.

5. Detalles de implementación

6. Detalle de Implementación

A continuación, se describen las estrategias de diseño e implementación utilizadas para resolver los puntos más conflictivos del sistema, justificando las decisiones arquitectónicas en base a patrones de diseño, principios SOLID y la elección entre herencia y delegación.

6.1. Partida

Para la clase `Partida` se aplicaron rigurosamente **Single Responsibility Principle (SRP)**, **Dependency Inversion Principle (DIP)** y el principio **Tell, Don't Ask (TDA)**.

Al ser la clase que orquesta el flujo global del juego, existía el riesgo de convertirla en una super clase con exceso de responsabilidades. Para mitigar esto, la evaluación de las condiciones de victoria se delegó al `EvaluadorGanador` mediante el método `resultado()`, el registro histórico se delegó a `RegistroPartida`, y el comportamiento del ciclo diurno/nocturno se encapsuló en la jerarquía `Fase`.

Asimismo, se aplica el patron `Factory`, donde `FabricaFases` actúa como creador de fases mediante métodos (`crearFaseNocturna` y `crearFaseDiurna`), manteniendo el control del ciclo de turno.

6.2. Registro de Eventos de la Partida

Para representar los distintos acontecimientos que ocurren durante una ronda se implementó la jerarquía `RegistroRonda`. Sus subclases (`RegistroNocturno`, `RegistroDiurno`, `RegistroBallotage`, etc.) redefinen polimórficamente el método `describir()`.

Justificación: En este escenario específico se optó por la herencia, ya que las variantes de los registros cumplen la relación "es un" heredan todos el contrato. La utilización de herencia y polimorfismo eliminó por completo el uso de estructuras condicionales, aplicando el principio **Open/Closed Principle (OCP)**, permitiendo agregar nuevos tipos de reportes sin alterar la lógica de (`RegistroPartida`).

6.3. Estado Dinámico de los Jugadores

El estado vital de un jugador se modeló mediante el patrón de diseño **State**, utilizando la clase abstracta `Estado` y sus implementaciones concretas `Vivo` y `Eliminado`.

Justificación: A diferencia del registro de eventos, aquí la herencia hubiese sido un error de diseño crítico. Si existe un `JugadorVivo` que hereda de una clase `Jugador`, su estructura sería estática y no podría pasar de vivo a eliminado de forma dinámica. Al utilizar delegación, la clase `Jugador` mantiene una referencia interna a un `Estado` mutable. Cuando el jugador es asesinado, simplemente se intercambia su instancia por una de tipo `Eliminado`. Esto respeta el principio de favorecer la composición sobre la herencia, reduce el acoplamiento y cumple con el **SRP**, ya que la clase `Jugador` se desentiende de las reglas algorítmicas de qué puede o no hacer un espectador eliminado.

6.4. Resolución de Empates y Consensos

Tanto en la votación de los ciudadanos (día) como en la de la mafia (noche) se identificó un punto conflictivo: las reglas para determinar el resultado de empate pueden variar drásticamente según el criterio o futuras expansiones del juego. Para resolver esto, se aplicó el patrón de diseño **Strategy**.

- **Fase Diurna:** La clase `VotacionDiurna` delega la resolución de empates a la interfaz `CriterioEmpate`, permitiendo intercambiar estrategias como `SinEliminacion` o `Ballotage`.

- **Fase Nocturna:** La clase `VotacionMafia` delega la búsqueda de la víctima a la interfaz `CriterioConsenso`, permitiendo alternar entre una estrategia de `Mayoria` o `SinVictima` (en caso de desacuerdo absoluto).

Justificación: Se prefirió la ****delegación**** porque permite parametrizar las votaciones con diferentes variantes en tiempo de configuración sin necesidad de crear subclases complejas como `VotacionDiurnaConBallotage`. Esto garantiza el cumplimiento del **OCP**, permitiendo añadir nuevos criterios de desempate creando una simple clase que implemente la interfaz correspondiente.

6.5. Bandos, Polimorfismo y Double Dispatch

La pertenencia a una facción se modeló mediante la interfaz `Bando` y sus implementaciones `BandoMafia` y `BandoCiudadano`, encapsulando las condiciones de victoria en el método `ganoSegun(jugadores)`.

Para resolver de manera limpia la comparación entre bandos y la lógica de investigación del `Detective` sin recurrir al operador `instanceof` (el cual viola el principio de diseño de diseño OCP y Liskov Substitution Principle), se implementó la técnica de **Double Dispatch** (Doble Despacho). A través de métodos cruzados como `esMismoBandoQueMafia(BandoMafia)` y `esMismoBandoQueCiudadano(BandoCiudadano)`, los objetos delegan la resolución del tipo concreto al mecanismo de despacho dinámico del propio lenguaje de programación. Esto permite que el sistema determine si dos jugadores pertenecen al mismo bando de forma puramente polimórfica y fuertemente tipada.

6.6. Roles mediante Composición

Los roles del juego se estructuraron bajo la jerarquía abstracta `Rol`, de la cual heredan clases de comportamiento especializado como `Mafioso`, `Padrino`, `Detective`, `Médico` y `Sheriff`.

Justificación: Si un jugador mafioso se modelara heredando directamente de una clase base `Jugador` (ej. `class JugadorMafioso extends Jugador`), el sistema sufriría de una explosión combinatoria de clases imposible de mantener al cruzarse con los estados (ej. `JugadorVivoMafioso`, `JugadorEliminadoDetective`). Al utilizar ****delegación por composición****, la clase `Jugador` simplemente “tiene un” `Rol`. Esto separa la identidad intrínseca del jugador (su nombre, su estado vital) de las capacidades y acciones nocturnas de su rol, reduciendo el acoplamiento y permitiendo que un `Padrino` implemente su propia variante prioritario de voto (`VotoPadrino`) o altere polimórficamente el método `resultadoAlSerInvestigado()` de manera aislada.

6.7. Configuración Inicial

La inicialización del juego se aislaron completamente del flujo de la partida mediante las clases `Configuracion`, `Tramo` y `RepartidorRoles`.

El uso de estas clases especializadas responde al principio de alta cohesión. `Configuracion` se encarga de definir cuántos mafiosos y roles especiales corresponden según el volumen de participantes basándose en objetos `Tramo`, y delega la aleatoriedad a la interfaz `MezcladorDeRoles` (aplicando nuevamente el patrón **Strategy** a través de `MezcladorAleatorioRoles`). Al extraer esta lógica condicional de la clase `Partida`, se asegura que el núcleo del juego reciba un conjunto de `Jugadores` ya listo y validado para operar.

6.8. Fases del Ciclo de Juego

El flujo temporal y la máquina de estados del juego aplicando el **Interface Segregation Principle** se modelaron mediante la abstracción `Fase` y sus dos concreciones: `FaseDiurna` y `FaseNocturna`.

Cada fase dependiendo de las operaciones permitidas en su contexto implementa a su interfaz correspondiente, `AccionNocturna` o `AccionDiurna`. Esta separación de comportamiento permite desacoplar responsabilidad innecesaria en cada fase.

6.9. Interfaz Gráfica y Desacoplamiento (Observer y Command)

El desarrollo de la interfaz gráfica (GUI) presentó el desafío técnico de permitir la interacción del usuario sin alterar el código del modelo con dependencias visuales. Para resolver este conflicto arquitectónico, se implementaron dos patrones de diseño fundamentales:

1. **Patrón Observer (Observador):** Se utilizó para establecer una comunicación unidireccional y desacoplada desde el Modelo hacia la Vista. Las clases del núcleo (como **Partida** o **GestorJuego**) implementan una interfaz **IObservable**, mientras que las ventanas y componentes visuales implementan **IObservador**. Cuando ocurre un cambio en el estado del juego (ej. un jugador es eliminado), el modelo simplemente invoca a **notificarObservadores()**. La vista reacciona de forma pasiva redibujando la pantalla, garantizando que el Modelo pueda funcionar de forma autónoma y “a ciegas” (por ejemplo, en pruebas unitarias o consola) sin conocer la existencia de botones o ventanas.
2. **Patrón Command (Comando):** Para el flujo inverso (de la Vista hacia el Modelo), se evitó que los botones ejecuten lógica de negocio directamente. En su lugar, cada interacción del usuario se encapsula en un objeto ejecutable que implementa la interfaz **IAccion** (como **AccionInvestigar** o **AccionPasarTurno**). Cuando el usuario presiona un botón, la Vista instancia el comando correspondiente y se lo envía al modelo.

Justificación de Delegación sobre Herencia en los Comandos: Los comandos actúan mediante delegación, empaquetando los parámetros requeridos (votante, objetivo) y abstrayendo la orden. Esto permite al modelo encolar las acciones nocturnas secretas de los jugadores y ejecutarlas en un orden estrictamente controlado al amanecer, facilitando la escalabilidad del sistema ante la futura adición de nuevos roles y habilidades sin modificar el despachador central de la partida.

7. Excepciones

En el desarrollo del trabajo se implementaron las siguientes excepciones propias, con el objetivo de representar y manejar situaciones de error específicas del dominio del sistema.

Estas excepciones fueron creadas para gestionar situaciones en las que se viola una precondición establecida por las reglas del juego o por la lógica del modelo. Su utilización se basa en la validación de dichas precondiciones dentro del código mediante estructuras condicionales, lanzando explícitamente la excepción correspondiente cuando la condición no se cumple.

Asimismo, en las pruebas unitarias y de integración se verifica su correcto funcionamiento mediante 'assertThrows', validando que las excepciones sean lanzadas en los casos esperados junto con sus mensajes asociados.

- **CantidadDeJugadoresInvalidaException:** se lanza cuando la configuración inicial de la partida recibe una cantidad de jugadores fuera del rango permitido. Se utiliza en la clase `Configuracion` mediante el método `orElseThrow` y se verifica en `TestConfiguracion` mediante `assertThrows`.
- **InvestigacionInvalidaException:** se lanza cuando el Detective intenta investigarse a sí mismo, investigar a un jugador eliminado o investigar al mismo jugador en turnos consecutivos. Del mismo modo, también se lanza cuando se intenta investigar fuera de la fase nocturna o cuando un jugador sin el rol correspondiente intenta realizar una investigación. Se utiliza en las clases `Investigador` y `Detective`, y se verifica en `TestDetective` y `TestJugador`. En la parte visual también se utiliza para advertir al jugador, por ejemplo, en las clases `AccionRolInvestigar` y `AccionInvestigar`.
- **JugadoresException:** se lanza cuando se intenta obtener los cómplices de un jugador que no pertenece al bando mafioso o que se encuentra eliminado. Se utiliza en las clases `Jugadores` y `TestJugadores`.
- **NominacionInvalidaException:** se lanza cuando una nominación no es válida, en particular cuando se intenta nominar a un jugador eliminado. También se lanza cuando se intenta nominar fuera de la fase diurna. Se utiliza en `VotacionDiurna`, `TestIntegrales` y `AccionNominar`.
- **ProteccionInvalidaException:** se lanza cuando el Médico intenta proteger a un jugador eliminado o al mismo jugador en turnos consecutivos. Además, también se lanza cuando un jugador que no posee el rol de Médico intenta proteger a otro jugador o cuando se intenta proteger fuera de la fase nocturna. Se utiliza en la clase `Medico`, en `TestMedico` y también en `AccionProteger` y `AccionRolProteger`.
- **RepartoRolesInvalidoException:** se lanza cuando la cantidad de roles a repartir no coincide con la cantidad de jugadores de la partida. Se utiliza en `RepartidorRoles`, `TestIntegrales` y `TestRepartidorRoles`.
- **RevelacionInvalidaException:** se lanza cuando se intenta revelar en fase nocturna o cuando el revelador no es el sheriff. Se utiliza en las clases `TestFaseDiurna`, `TestSheriff` y en las clases de Acciones tal como `AccionRolRevelarInvestigado`, `AccionRevelarSheriff` y `AccionRolRevelarse`.
- **VotacionInvalidaException:** se lanza cuando se intenta emitir un voto inválido. En la votación diurna, esto ocurre al votar por un jugador no nominado o eliminado. En la votación de la Mafia, se produce al intentar votar por una víctima no válida o por un miembro de la propia Mafia. Se utiliza en `VotacionDiurna`, `VotacionMafia`, sus respectivos tests y las correspondientes clases controladoras de la interfaz.